

National University of Computer and Emerging Sciences



Introduction to Data Science Lab

Instructor: Mariam Nasim

Objectives

After performing this lab, students shall be able to understand implementation of linear regression:

- ✓ From scratch
- ✓ Using built-in libraries

Regression from scratch:

Linear regression tries to model the relationship between:

- Input (independent variable) X
- Output (dependent variable) Y

Using the equation of a straight line:

$$Y = B_0 + B_1 \cdot X$$

Where:

- B_0 = Intercept
- B_1 = Slope

◆ Step 1: Prepare the Data

Ensure your input data is:

- Numeric and clean (no missing values)
- In two 1D lists or arrays:
 - X : independent variable (input)
 - Y : dependent variable (target/output)
- Both lists/arrays are the **same length**

◆ Step 2: Calculate the Coefficients

We use the **least squares method** to find:

- Slope B_1 :

$$B_1 = \frac{n \sum XY - \sum X \sum Y}{n \sum X^2 - (\sum X)^2}$$

- Intercept B_0 :

$$B_0 = \bar{Y} - B_1 \cdot \bar{X}$$

✚ Compute:

- Mean of `x` and `y`
- The numerator and denominator for `B1`
- Plug values into the formulas

◆ Step 3: Make Predictions

Once you have B_0 and B_1 , use the regression formula:

$$\hat{Y} = B_0 + B_1 \cdot X$$

Loop through your `x` values to generate predicted `y` values.

◆ Step 4: Evaluate the Model (Optional)

Check how well your model fits the data.

🔧 Metrics:

- Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum (Y - \hat{Y})^2$$

- R-squared (R^2):

$$R^2 = 1 - \frac{\sum (Y - \hat{Y})^2}{\sum (Y - \bar{Y})^2}$$

◆ Step 5: Visualize the Results (Optional)

Plot the data and the regression line using `matplotlib`.

Regression using built-in libraries

A general flow of implementing regression is given below:

◆ 1. Import Libraries

```
python

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

◆ 2. Load the Dataset

```
python

df = pd.read_csv('your_dataset.csv') # Or any data source
print(df.head())
```

◆ 3. Explore and Preprocess the Data

- Handle missing values
- Convert categorical to numeric if needed
- Drop irrelevant columns

```
python

df = df.dropna() # or df.fillna()
# df['Category'] = df['Category'].map({'A': 0, 'B': 1})
```

◆ 4. Define Features and Target

python

```
X = df[['feature1', 'feature2']] # Independent variables
y = df['target']                 # Dependent variable
```

◆ 5. Split into Train and Test Sets

python

📄 Copy

✎ Edit

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

◆ 6. Train the Regression Model

python

```
model = LinearRegression()
model.fit(X_train, y_train)
```

◆ 7. Make Predictions

python

```
y_pred = model.predict(X_test)
```

◆ 8. Evaluate the Model

python

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R-squared Score:", r2_score(y_test, y_pred))
```

Lab Tasks:

Task 1:

1. Download the csv file provided.
2. Load data into a pandas dataframe.
3. Extract columns as NumPy arrays
`hours = df['hours'].values`
4. Create the following functions: (use numpy functions for sum and mean calculations)
 - `calculate_coefficients(X, Y)`
 - **Inputs:** Lists or arrays x (independent variable), y (dependent variable)
 - **Purpose:** Calculate the **slope (B1)** and **intercept (B0)** using the formulas:

$$a = \frac{\sum X^2 \sum Y - \sum X \sum XY}{n \sum X^2 - (\sum X)^2}$$

$$B = \frac{n \sum XY - \sum X \sum Y}{n \sum X^2 - (\sum X)^2}$$

- **Returns:** Intercept (B0) and Slope (B1)
-
- `predict(X, B0, B1)`
 - **Inputs:** List of X values, and the coefficients B0 and B1
 - **Purpose:** Compute predicted values using the regression line formula:
- $$\hat{Y} = B_0 + B_1 \cdot X$$
- **Returns:** List of predicted values
5. Compute the slope and intercept from the given data.
 6. Using your `predict()` function and calculated coefficients, predict:
 - Test score for 5.5 hours
 - Test score for 9 hours
 7. Example output:

Intercept (B0): 30.00

Slope (B1): 6.00

Prediction for 5.5 hours: 63.00

Prediction for 9 hours: 84.00

Task 2:

- 1) Use LinearRegression from sklearn.linear_model to train a model on the same dataset.
- 2) Print the slope and intercept from the built-in model. {model.coef_[0]} {model.intercept_}
- 3) Predict test scores using the trained LinearRegression model for:
 - 5.5 hours
 - 9 hours

Task 3:

Compare the coefficients and predictions from both models:

How close are the slope and intercept values?

Are the predictions very similar or noticeably different?

Task 4: Visualization

- 1) Plot the original data points using a scatter plot.
- 2) Plot both regression lines (from-scratch and scikit-learn) on the same graph. Use different colors.

Task 5:

- 1) Split the dataset into training and testing sets.

Use **scikit-learn's** `train_test_split` directly on the DataFrame or on the extracted arrays

```
from sklearn.model_selection import train_test_split

# Assuming df is your DataFrame
X = df[['hours']] # Keep as DataFrame or convert to numpy array
y = df['scores']

# Split into train and test sets (e.g., 70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

- 2) Train a LinearRegression model on the training set using builtin sklearn function (as done in task 2).
- 3) Predict test scores on the test set.
- 4) Calculate and print the Mean Squared Error (MSE) of predictions on the test set.

Submission Instructions

- Submit a Jupyter Notebook (.ipynb) or Python script with all code and answers.